UNIVERSIDADE FEDERAL DE SERGIPE

Design Patterns

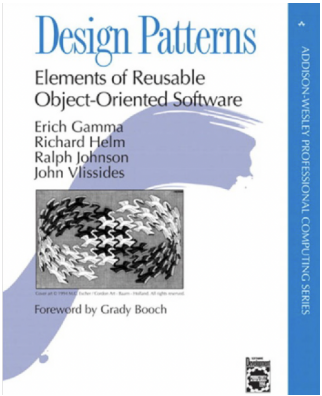
Programação Orientada a Objetos

Prof. Dr. André Yoshiaki Kashiwabara

Universidade Federal de Sergipe

January 18, 2026

Design Patterns Elements of Reusable Object-Oriented Software



O Desafio do Design Orientado a Objetos

- **A Realidade:** Projetar software OO é difícil. Torná-lo reutilizável é ainda mais difícil.
- **Tarefas do Arquiteto:**
  - Encontrar objetos pertinentes.
  - Fatorar em classes com a granularidade correta.
  - Definir interfaces e hierarquias de herança.
  - Estabelecer relacionamentos-chave.
- **O Dilema:** O design deve ser específico para o problema atual, mas geral para o futuro.
- **Iteração:** Raramente se acerta na primeira vez. O reuso e a modificação refinam o design.

Novatos vs. Especialistas

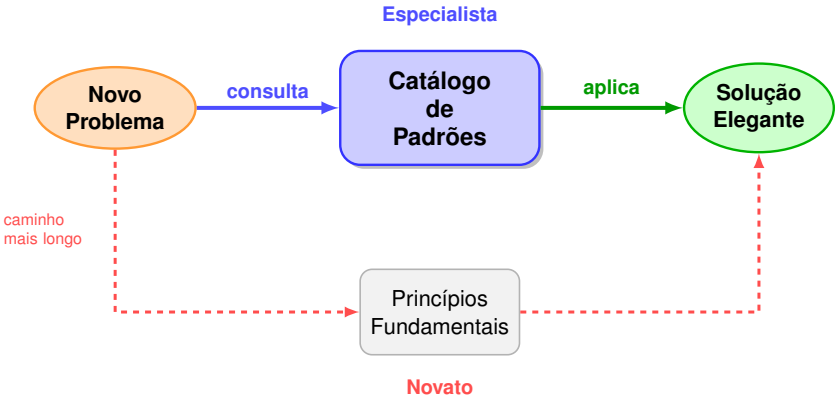
O Problema dos Novatos

Muitas opções disponíveis geram sobrecarga, levando ao uso de técnicas não orientadas a objetos por hábito.

O Segredo dos Especialistas

- Não resolvem problemas a partir dos princípios fundamentais (do zero).
- Reutilizam soluções que funcionaram no passado.
- Identificam **padrões recorrentes** de classes e comunicação de objetos.

O Ciclo de Reutilização de Experiência



Por que Design Patterns?

- **Registro de Experiência:** Captura designs recorrentes e comprovados (evita o "déjà-vu de design" sem solução).
- **Acessibilidade:** Torna técnicas avançadas acessíveis a novos desenvolvedores.
- **Comunicação:** Fornece um vocabulário comum e melhora a documentação do sistema.
- **Velocidade:** Ajuda a acertar o design mais rápido, focando na reutilização e manutenibilidade.

Escopo e Limitações

Não são designs novos

Os padrões descritos são parte do folclore ou de sistemas de sucesso existentes. São designs **comprovados**.

O que este catálogo NÃO cobre

- Concorrência ou sistemas distribuídos.
- Programação em tempo real.
- Interfaces de usuário (UI) específicas.
- Drivers de dispositivos ou Bancos de Dados OO.

Focamos em capturar a experiência essencial de design de objetos em um formato consistente.

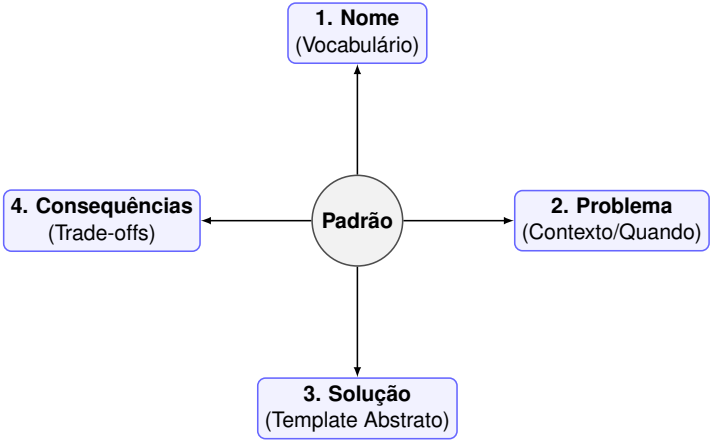
A Definição de Padrão (Christopher Alexander)

Definição Core

"Cada padrão descreve um problema que ocorre repetidamente em nosso ambiente e, então, descreve o núcleo da solução para esse problema..."

- **Origem:** Arquitetura de edifícios e cidades.
- **Paralelo com Software:**
  - Paredes e Portas $\rightarrow$ Objetos e Interfaces.
  - Essência: Solução para um problema em um contexto.

Os 4 Elementos Essenciais de um Padrão



Detalhando os Elementos (1 e 2)

- **1. O Nome do Padrão**
  - Cria um "handle" (identificador) para o problema/solução.
  - Eleva o nível de abstração do design.
  - Facilita a comunicação entre equipe e documentação.

- **2. O Problema**
  - Define *quando* aplicar o padrão.
  - Descreve sintomas de design inflexível.
  - Pode listar pré-condições necessárias.

Detalhando os Elementos (3 e 4)

- **3. A Solução**
  - Não é uma implementação concreta (código pronto).
  - É um *template* abstrato aplicável em várias situações.
  - Define relacionamentos, responsabilidades e colaborações entre classes.

- **4. As Consequências**
  - Resultados e *trade-offs* (compromissos) da aplicação.
  - Aspectos críticos: Espaço, Tempo, Flexibilidade, Extensibilidade.
  - Ajuda a avaliar custos vs. benefícios.

O Nível de Abstração dos Design Patterns



Design Patterns não são blocos primitivos (listas) nem sistemas inteiros.

Smalltalk MVC: Uma Fonte de Padrões

- **Contexto:** Usado para construir interfaces de usuário no Smalltalk-80.
- **A Tríade:**
  1. **Model:** Objeto da aplicação (Dados).
  2. **View:** Apresentação na tela.
  3. **Controller:** Reação à entrada do usuário.
- **Objetivo Histórico:** Antes do MVC, esses objetos eram misturados. O MVC os **desacopla** para aumentar flexibilidade e reuso.

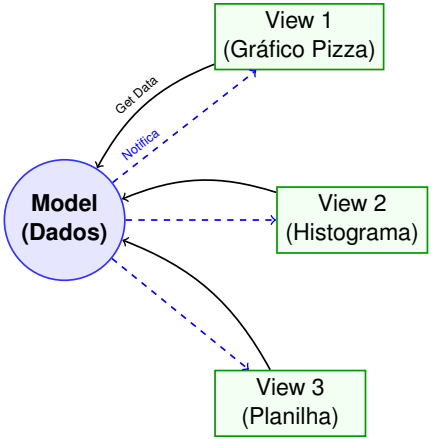
Model e View: O Padrão Observer

O Problema

Como permitir que múltiplas telas (gráficos, planilhas) reflitam os mesmos dados sem que os dados conheçam os detalhes das telas?

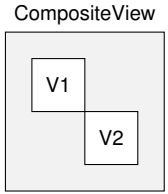
- **Protocolo Assinar/Notificar:** O Model notifica as Views quando os dados mudam.
- **Atualização:** As Views buscam os novos dados e se redesenham.
- **Resultado:** Desacoplamento. Mudanças em um objeto afetam N outros dinamicamente.

Visualizando o Observer no MVC



Views Aninhadas: O Padrão Composite

- Views podem conter outras views (ex: Painel de Controle com Botões).
- **Classe CompositeView:** Subclasse de View que gerencia filhos.
- **Conceito Chave:** Tratar um grupo de objetos (composto) da mesma forma que um objeto individual (primitivo).
- **Padrão Composite:** Cria hierarquias parte-todo recursivas.



View e Controller: O Padrão Strategy

Flexibilidade na Entrada

Como mudar a resposta ao usuário (ex: Teclado vs. Menu) sem mudar a apresentação visual?

- **Encapsulamento:** O MVC encapsula a resposta em um objeto *Controller*.
- **Substituição:** Uma View usa uma instância de Controller. Trocar o controlador troca o comportamento.
- **Padrão Strategy:** Define uma família de algoritmos (estratégias de resposta), encapsula cada um e os torna intercambiáveis.

Resumo: O Ecossistema MVC

O MVC não é apenas um padrão, é uma coleção deles trabalhando juntos:

Observer:	Desacopla Model de Views (Sincronização de estado).
Composite:	Permite Views aninhadas e hierarquias de interface.
Strategy:	Permite trocar o comportamento de entrada (Controller).

- *Outros mencionados:* Factory Method (criação padrão), Decorator (adicionar scroll).

Por que um Formato Padronizado?

Limitações das Notações Gráficas

Diagramas (UML/OMT) mostram apenas o **produto final** (classes e relacionamentos).

- Para reutilizar um design, precisamos entender:
  - As decisões tomadas.
  - As alternativas consideradas.
  - Os *trade-offs* (compromissos).
- **Solução:** Um template consistente (texto + diagrama + código).
- **Benefício:** Facilita o aprendizado, a comparação e o uso.

O Template: Identificação e Contexto

Nome e Classificação
: Essência do padrão. Vital para o vocabulário de design.

Intenção
: O que o padrão faz? Qual problema ele resolve? (Resumo executivo).

Também Conhecido Como
: Sinônimos ou apelidos do padrão.

Motivação
: Um cenário concreto de problema/solução para facilitar o entendimento antes da abstração.

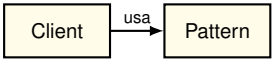
Aplicabilidade
: Quando usar? Como reconhecer situações que requerem este padrão?

O Template: Estrutura e Dinâmica

Estrutura
: Representação gráfica (OMT) das classes.

Participantes
: Classes/Objetos e suas responsabilidades.

Colaborações
: Como os participantes interagem.



"A seção Estrutura e Participantes define quem é quem."

O Template: Consequências e Código

Consequências (Crítico)

- Como o padrão atinge seus objetivos?
- Quais são os **trade-offs** (custos vs. benefícios)?
- O que pode variar independentemente?

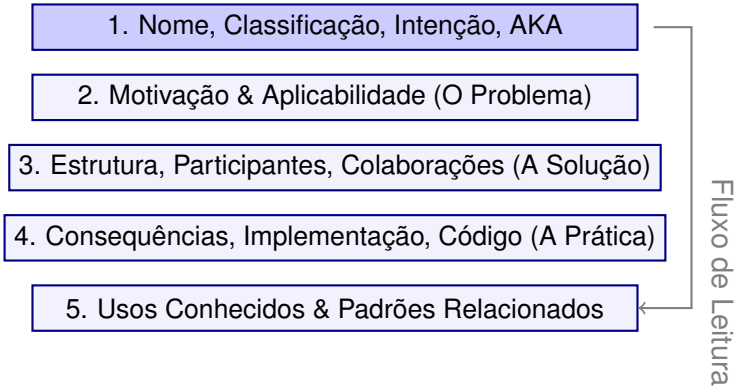
Implementação Dicas, armadilhas e especificidades de linguagem.

Código de Exemplo Fragmentos reais (C++ ou Smalltalk).

O Template: Contexto e Referências

- **Usos Conhecidos:**
  - Exemplos do mundo real (pelo menos dois domínios diferentes).
  - Valida que o padrão é recorrente e útil.
- **Padrões Relacionados:**
  - Quais padrões trabalham bem juntos?
  - Quais são as diferenças sutis entre padrões similares?

Resumo Visual do Template



O Catálogo de Padrões de Projeto

- O catálogo do Livro apresenta **23 Padrões de Projeto**. Atualmente, há outros padrões que foram identificados.
- Organizados para resolver problemas recorrentes de design.
- A seguir, apresentamos uma visão geral dividida por categoria (Criação, Estrutural e Comportamental).

Padrões de Criação (Creational)

Abstract Factory	Interface para criar famílias de objetos relacionados sem especificar classes concretas.
Builder	Separa a construção de um objeto complexo da sua representação.
Factory Method	Define interface de criação, mas deixa subclasses decidirem qual classe instanciar.
Prototype	Cria novos objetos copiando uma instância prototípica.
Singleton	Garante que uma classe tenha apenas uma instância e fornece acesso global a ela.

Padrões Estruturais (Structural) - I

Adapter	Converte a interface de uma classe em outra esperada pelos clientes.
Bridge	Desacopla uma abstração de sua implementação (podem variar independentemente).
Composite	Compõe objetos em árvores para representar hierarquias parte-todo (tratamento uniforme).
Decorator	Anexa responsabilidades adicionais a um objeto dinamicamente (alternativa à herança).

Padrões Estruturais (Structural) - II

Facade	Fornece uma interface unificada (alto nível) para um conjunto de interfaces em um subsistema.
Flyweight	Usa compartilhamento para suportar muitos objetos de granulação fina eficientemente.
Proxy	Fornece um substituto (placeholder) para controlar o acesso a outro objeto.

Padrões Comportamentais (Behavioral) - I

Chain of Responsibility	Passa uma solicitação por uma cadeia de objetos até que um a trate.
Command	Encapsula uma solicitação como objeto (fila, log, undo).
Interpreter	Define uma representação gramatical e um interpretador para uma linguagem.
Iterator	Acessa elementos de um agregado sequencialmente sem expor a representação.
Mediator	Encapsula como um conjunto de objetos interage (promove baixo acoplamento).

Padrões Comportamentais (Behavioral) - II

Memento	Captura o estado interno de um objeto para restauração futura (sem violar encapsulamento).
Observer	Dependência um-para-muitos: quando um muda, notifica os dependentes.
State	Permite que um objeto altere seu comportamento quando seu estado interno muda.
Strategy	Define uma família de algoritmos intercambiáveis.
Template Method	Define o esqueleto de um algoritmo, adiando passos para subclasses.
Visitor	Define uma nova operação sem mudar as classes dos elementos onde opera.

Organizando o Catálogo

- Padrões variam em **granularidade** e **abstração**.
- **Por que classificar?**
  - Facilita referenciar famílias de padrões.
  - Acelera o aprendizado do catálogo.
  - Ajuda na descoberta de novos padrões.
- **Dois Critérios Principais:**
  1. **Propósito:** O que o padrão faz (Criação, Estrutural, Comportamental).
  2. **Escopo:** A quem o padrão se aplica (Classe ou Objeto).

Critério 1: Propósito

Reflete o objetivo fundamental do padrão:

De Criação (Creational)

Processo de criação de objetos.

Estrutural (Structural)

Composição de classes ou objetos.

Comportamental (Behavioral)

Interação e distribuição de responsabilidades.

Critério 2: Escopo (Classe vs. Objeto)

Padrões de Classe

- Focam em **Classes e Subclasses**.
- Mecanismo: **Herança**.
- Característica: **Estáticos** (Tempo de Compilação).

Padrões de Objeto

- Focam em **Relacionamento entre Objetos**.
- Mecanismo: **Composição**.
- Característica: **Dinâmicos** (Tempo de Execução).

Nota: A maioria dos padrões no catálogo são de Escopo de Objeto.

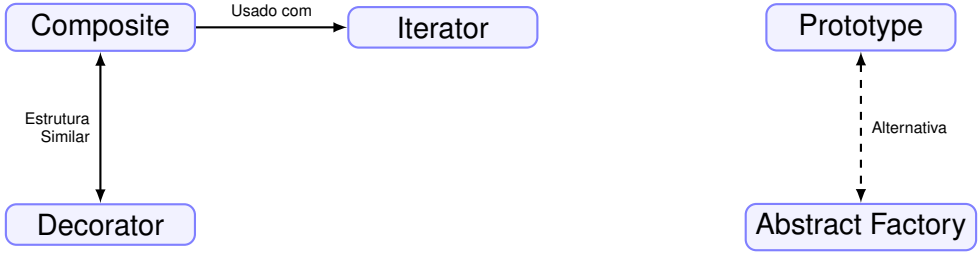
A Matriz de Classificação (Propósito × Escopo)

	Criação	Estrutural	Comportamental
Classe	Adia a criação para **subclasses** `Factory Method`	Usa **herança** para compor classes `Adapter`	Herança para algoritmos e controle `Template Method`

Objeto	Adia criação para outro **objeto**
	`Abstract Factory`
		Montagem de objetos
		`Composite`
		`Decorator`
			Cooperação entre grupo de objetos
			`Strategy`
			`Observer`

Outras Formas de Organização e Relacionamentos

Além da matriz Propósito/Escopo, os padrões se relacionam de outras formas:



- **Usados Juntos:** Combinam forças para resolver problemas complexos.
- **Alternativas:** Diferentes soluções para o mesmo problema.
- **Similaridade:** Estruturas parecidas, intenções diferentes.

Como Padrões Resolvem Problemas: Encontrando Objetos

- **Definição Básica:** Objetos empacotam dados e procedimentos (métodos).
- **Encapsulamento:**
  - Solicitações (mensagens) são a única forma de executar operações.
  - Operações são a única forma de alterar dados internos.
- **O Grande Desafio:** Decompor um sistema em objetos é difícil.
- **Fatores Conflitantes:**
  - Encapsulamento, Granularidade, Dependência.
  - Flexibilidade, Desempenho, Evolução, Reutilização.

Abordagens Tradicionais de Modelagem

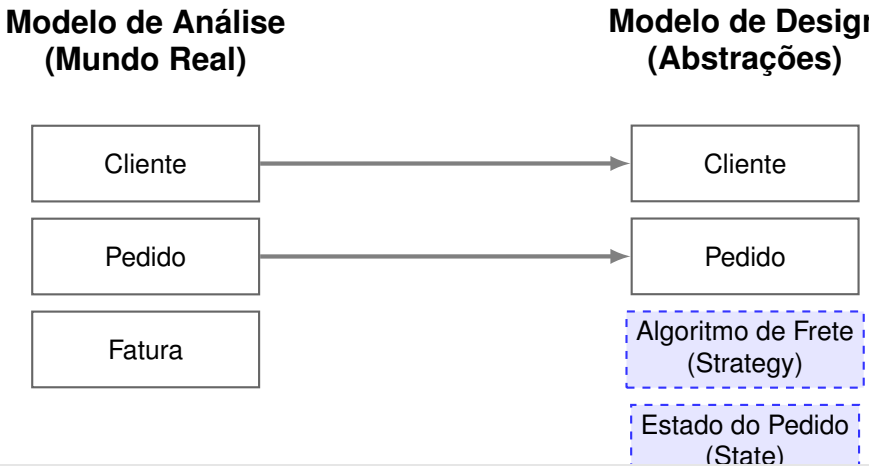
Não existe consenso sobre a "melhor" maneira de encontrar objetos:

- **Análise Linguística:** Destacar substantivos (classes) e verbos (operações) na descrição do problema.
- **Foco em Colaboração:** Identificar responsabilidades e interações no sistema (CRC Cards).
- **Modelagem do Mundo Real:** Traduzir entidades físicas da análise diretamente para o código.

O Perigo da Modelagem Estrita

Modelar apenas o mundo real reflete a realidade de *hoje*, mas não necessariamente a de *amanhã*.

Além do Mundo Real: Objetos de Design



Como Padrões Ajudam a Encontrar Objetos

Padrões de projeto revelam **abstrações menos óbvias** que tornam o sistema flexível:

Strategy (315)

- Representa um *algoritmo* ou *processo* como um objeto.
- Permite trocar algoritmos (famílias intercambiáveis).
- Não existe "na natureza".

State (305)

- Representa o *estado* de uma entidade como um objeto.
- Permite mudar comportamento dinamicamente.
- Descoberto ao buscar flexibilidade, não na análise inicial.

Padrões fornecem objetos que capturam conceitos abstratos, não apenas substantivos físicos.

Determinando a Granularidade do Objeto

- **A Variedade:** Objetos variam de representações de hardware a aplicações inteiras.
- **O Problema:** Como decidir o "tamanho" correto de um objeto?
- **Soluções nos Padrões (Extremos):**

  Macro (Facade) Representa *subsistemas inteiros* como um único objeto para facilitar o uso.

  Micro (Flyweight) Suporta *enormes quantidades* de objetos de granularidade finíssima (compartilhamento de estado).

Decomposição Funcional em Objetos Menores

Outros padrões focam em extrair responsabilidades específicas para novos objetos menores:

Objetos de Criação

- **Abstract Factory & Builder:**
- A única responsabilidade é *criar* outros objetos (retiram essa lógica do objeto principal).

Objetos de Ação/Solicitação

- **Visitor & Command:**
- A única responsabilidade é *implementar uma solicitação* em outro objeto ou grupo.

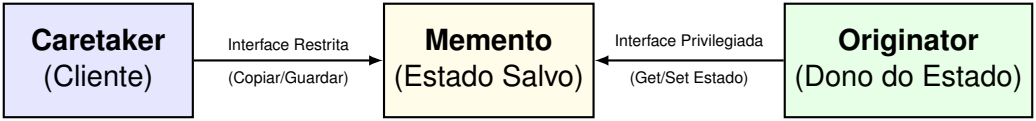
Como Padrões Ajudam a Definir Interfaces

Padrões não apenas usam interfaces, eles ditam o que deve (ou não) estar nelas.

Exemplo: O Padrão Memento (283)

- **Problema:** Salvar estado sem quebrar encapsulamento.
- **Solução de Interface Dupla:**
  1. **Interface Restrita (Narrow):** Para o cliente (apenas segurar o objeto).
  2. **Interface Privilegiada (Wide):** Para o criador (acesso aos dados internos).

Visualizando Interfaces no Memento



Restrições e Relacionamentos entre Interfaces

Alguns padrões impõem igualdade ou estrutura estrita nas interfaces:

Identidade de Interface

Decorator (175) & Proxy (207)

- A interface do Decorator/Proxy deve ser **idêntica** à do objeto original.
- Permite substituição transparente.

Reflexo de Classes

Visitor (331)

- A interface do Visitor deve refletir todas as classes concretas que podem ser visitadas.
- Ex: `visitConcreteElementA()`, `visitConcreteElementB()`.

Mecanismos de Reutilização

- **O Desafio:** Aplicar conceitos básicos (objetos, classes) para criar software flexível.
- **Duas Técnicas Principais:**
  1. **Herança de Classe:** Define a implementação de uma classe em termos de outra.
  2. **Composição de Objetos:** Obtém funcionalidade montando objetos (referências).

Caixa Branca vs. Caixa Preta

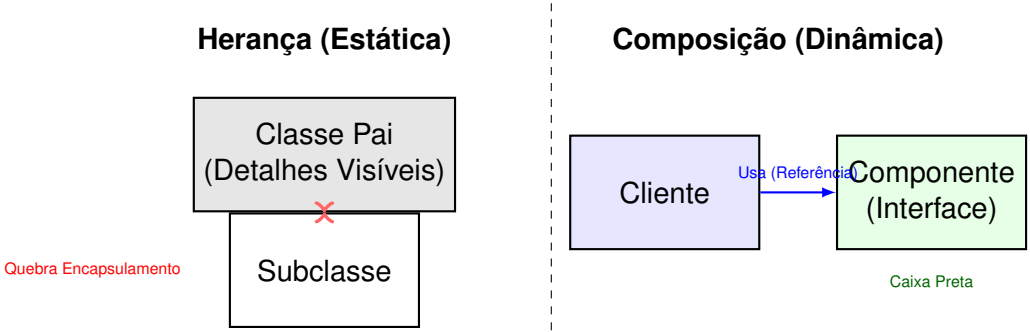
Herança (White-Box Reuse)

- **Visibilidade:** Internos da classe pai são visíveis para a subclasse.
- "Herança quebra o encapsulamento".

Composição (Black-Box Reuse)

- **Visibilidade:** Detalhes internos escondidos.
- Objetos aparecem como "caixas pretas".
- Exige interfaces bem definidas.

Visualizando o Acoplamento



Analisando a Herança

Vantagens

- Suportada diretamente pela linguagem (fácil de usar).
- Fácil de modificar a implementação reutilizada (sobrescrita).

Desvantagens

- **Estática:** Definida em tempo de compilação (não muda em runtime).
- **Dependência:** Mudanças no pai forçam mudanças no filho.
- Limita a flexibilidade e reutilização (dependência da implementação física).

Analisando a Composição

Vantagens

- **Dinâmica:** Definida em tempo de execução (objetos substituíveis).
- **Encapsulamento:** Respeita interfaces, mantém classes focadas.
- Menos dependências de implementação.

Efeitos no Design

- Hierarquias de classes menores e mais gerenciáveis.
- Mais objetos no sistema (o comportamento depende das relações).

O Segundo Princípio de Design OO

A experiência mostra que a herança é frequentemente usada em excesso. Projetos tornam-se mais simples e reutilizáveis quando dependem mais da composição.

"Prefira a composição de objetos à herança de classes."

Como Selecionar um Padrão de Projeto

- **O Desafio:** Com mais de 20 padrões, qual escolher?
- Se o catálogo é novo para você, a escolha não é óbvia.
- O livro propõe **6 abordagens** para filtrar e encontrar a solução correta.

Estratégias de Seleção (1 a 3)

1. **Considere como resolvem problemas de design:**
   - Volte à Seção 1.6 (Encontrar objetos, Granularidade, Interfaces).
   - Identifique qual desses desafios é o seu gargalo atual.

2. **Examine as Intenções (Scan Intents):**
   - Leia o resumo de "Intenção" de cada padrão.
   - Use a classificação (Criação, Estrutural, Comportamental) para filtrar.

3. **Estude as Inter-relações:**
   - Observe os gráficos de relacionamentos entre padrões.
   - Frequentemente, um padrão leva a outro complementar.

Estratégias de Seleção (4 a 6)

4. **Estude padrões de propósito semelhante:**
   - Leia as seções de "Comparação" no final dos capítulos.
   - Entenda as diferenças sutis entre padrões da mesma família (ex: Proxy vs. Decorator).

5. **Examine causas de Reprojeto (Redesign):**
   - O que está causando dor ou rigidez no sistema atual?
   - Mapeie o problema (ex: dependência de hardware) para a solução.

6. **Considere o que deve ser variável:**
   - O que você quer mudar sem quebrar o resto?
   - *"Encapsule o conceito que varia."*

Foco no que Varia (Encapsulamento)

Esta é talvez a abordagem mais poderosa:

Princípio Fundamental

Em vez de focar no que causa o erro, foque no que você deseja que seja flexível.

- **Algoritmo variando?** → Strategy.
- **Ação do usuário variando?** → Command.
- **Interface variando?** → Adapter.
- **Estrutura de objeto variando?** → Composite.

Identificar o "Eixo de Variação" leva diretamente ao padrão correto.

Como Usar um Padrão de Projeto

Uma vez escolhido o padrão, o processo de aplicação segue 5 passos fundamentais para garantir eficácia:

1. Leitura Panorâmica (Overview).
2. Análise Estrutural.
3. Estudo do Código de Exemplo.
4. Nomenclatura Contextualizada.
5. Definição das Classes (Implementação).

Fase 1: Compreensão e Análise

1. Visão Geral

   Leia o padrão todo. Foque em **Aplicabilidade** e **Consequências**.
   - *Pergunta:* "Este padrão realmente resolve meu problema sem criar novos?"

2. Estrutura e Relacionamentos

   Estude **Estrutura, Participantes e Colaborações**.
   - Entenda como as classes se relacionam (quem herda de quem, quem contém quem).

3. Código de Exemplo

   Estude a implementação concreta para entender a tradução do diagrama para a linguagem.

Fase 2: Adaptação (Nomenclatura)

4. Escolha Nomes Significativos

Os nomes originais do padrão (ex: *Strategy, Context, ConcreteStrategyA*) são abstratos demais.

- Adapte para o contexto da sua aplicação.
- **Dica:** Incorpore o nome do participante do padrão ao nome da classe para tornar o design explícito.

Exemplo: Padrão Strategy

Em vez de chamar apenas de `Layout`, use:

- `SimpleLayoutStrategy`
- `TeXLayoutStrategy`

Fase 3: Implementação

5. Defina as Classes

É a hora de escrever o código final.

- Declare as Interfaces.
- Estabeleça a Herança.
- Defina variáveis de instância (dados e referências).

Integração: Identifique classes *existentes* que serão afetadas e refatore-as para acomodar o padrão.

Resumo Visual do Processo

